Hybrid Topic : **GraphQL**

- **Door**: Silke Goris
- **Demo datum**: 19/03/2026
- **Code url github**:
  - https://github.com/silkegoris/silke-goris-hybrid-graphql

1 Korte uitleg topic

We maken een kleine Movie Database webapplicatie waarin films kunnen worden opgevraagd via GraphQL. In plaats van een klassieke REST API gebruiken we GraphQL, zodat de frontend exact kan kiezen welke data van een film wordt opgevraagd (bijvoorbeeld enkel de titel en het jaartal).

De applicatie zal bestaan uit:

- GraphQL API
- React/Next.js frontend (minimal)
- lijst met films die via GraphQL worden opgevraagd

De gebruiker kan bijvoorbeeld een query uitvoeren om alle films op te halen met hun titel en releasejaar.

1.1 Korte beschrijving

GraphQL is een querytaal voor APIs die ontwikkeld werd door Meta. Met GraphQL kan een client zelf bepalen welke data hij nodig heeft. In tegenstelling tot een REST API krijgt de client dus geen vaste response, maar kan hij zelf een query schrijven.

```
const typeDefs = gql`
  type Movie {
    id: ID
    title: String
    year: Int
  }

  type Query {
    movies: [Movie]
    movie(id: ID!): Movie
  }
`;
```

De server stuurt dan enkel deze velden terug.

1.2 Prerequisites

Wat moet al gekend zijn voor je aan dit topic begint:

- Basis JavaScript, React, Next.JS

Welke tools moeten geinstalleerd zijn voor je aan dit topic kan beginnen:

- Node.js, Next.js, Apollo Server, Apollo Client (voor mijn demo specifiek)

Info om deze tools te installeren (als het gaat om iets dat we **niet** in de lessen gezien hebben):

- Je moet sowieso Node.js geïnstalleerd hebben, en een project in React klaar hebben staan. Daarna kan je dan GraphQL installeren. Veel meer is er niet nodig.

1.3 Stappen

1. We beginnen met het aanmaken van een nieuw Next.JS project en starten de server. Je mag op alles "ja" zeggen qua opties.

```
npx create-next-app movie-graphql
npm run dev
```

2. Dan gaan we GraphQL en Apollo installeren via een simpel bash commando.

```
npm install graphql @apollo/server @apollo/client
```

3. We maken een GraphQL Api route aan, dit doen we in de volgende directory:

```
src/app/api/graphql/route.ts
```

4. Daarna definiëren we een simpel schema waaraan we een kleine dataset toevoegen, je kan ook zelf kiezen hoeveel films je erin zet. We voegen ook een resolver toe.

```
import { ApolloServer } from "@apollo/server";
import { startServerAndCreateNextHandler } from "@as-integrations/next";
import { gql } from "graphql-tag";
import { NextRequest } from "next/server";

const movies = [
  { id: "1", title: "Inception", year: 2010 },
  { id: "2", title: "Interstellar", year: 2014 },
];

const typeDefs = gql`
  type Movie {
    id: ID
    title: String
    year: Int
  }
  type Query {
    movies: [Movie]
    movie(id: ID!): Movie
  }
`;

const resolvers = {
  Query: {
    movies: () => movies,
    movie: (_: any, args: any) =>
      movies.find((m) => m.id === args.id),
  },
};

export const runtime = "nodejs";
const server = new ApolloServer({
  typeDefs,
  resolvers,
});
const apolloHandler = startServerAndCreateNextHandler(server);
export async function GET(request: NextRequest) {
  return apolloHandler(request as any);
}

export async function POST(request: NextRequest) {
  return apolloHandler(request as any);
}
```

5. We moeten daarna eerst ook de Apollo client nog toevoegen aan ons project.

src/lib/apolloClient.ts

```
import { ApolloClient, InMemoryCache } from "@apollo/client";

export const client = new ApolloClient({
  link: new HttpLink({
    uri: "/api/graphql",
  }),
  cache: new InMemoryCache(),
});
```

6. Daarna gaan we dan ook een voorbeeldpagina aanmaken zodat we kunnen testen of we onze data te zien krijgen. Dit is een hele simpele UI pagina.

src/app/movies/page.tsx

```
"use client";

import { gql, useQuery, ApolloProvider } from "@apollo/client";
import { client } from "@/lib/apolloClient";

const GET_MOVIES = gql`
  query {
    movies {
      id
      title
      year
    }
  }
`;

function MovieList() {
  const { loading, error, data } = useQuery(GET_MOVIES);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error</p>;

  return (
    <ul>
      {data.movies.map((movie: any) => (
        <li key={movie.id}>
          {movie.title} ({movie.year})
        </li>
      ))}
```

```
    </ul>
  );
}

export default function Page() {
  return (
    <ApolloProvider client={client}>
      <MovieList />
    </ApolloProvider>
  );
}
```

Wanneer we naar http://localhost:3000/api/graphql navigeren, zien we ook onze query editor. We kunnen daar queries genereren maar ook uitvoeren.

1.4 Referenties

- https://graphql.org/
- https://www.apollographql.com/docs/apollo-server
- https://www.apollographql.com/docs/react
- https://nextjs.org/docs

2 Conclusie

Zelf vond ik het redelijk makkelijk om met GraphQL te leren werken. In het begin was het wel even zoeken hoe de queries exact in elkaar zitten qua syntax, maar eens ik dit wat gewoon was, ging het redelijk vlot. Het heeft mij dan ook wel geholpen dat ik op mijn stageplaats al met GraphQL had gewerkt.